

Driver Fatigue Detection System with Gesture-Based Activation

Fairouz Abou Mousa, Qamar Aftan, Daniel Aguilar, Sami Akhnoukh, Mona Almadi

IE University

AI: Computer Vision

Professor ADRIAN CARRIO FERNANDEZ

1. Introduction:

Our project focuses on driver fatigue detection, as it is an important problem and a major cause of accidents worldwide. By monitoring the driver's behaviour and physical signs of fatigue, such as eye closure, yawning, and head movements, it is possible to detect early signs of drowsiness and help prevent accidents. In this project, we designed and implemented a computer vision system that detects driver fatigue in real time.

2. System Overview:

The system works by capturing video input from a camera, which is first used to detect a gesture-based activation sequence. Once activated, the system analyzes the driver's facial features to detect signs of fatigue such as eye closure, yawning, and head movement. If fatigue is detected, an alert is displayed to warn the driver.

3. Development Process:

The development of this project started by detecting driver fatigue using computer vision techniques while ensuring that the system only activates when needed through gesture-based control. Since we did not have direct access to a car for all group members, the data collection process was carried out by one of the group members with the help of family members. A custom dataset was created, consisting of both images and videos for gesture recognition and fatigue detection. The dataset contains a total of **2,169** files, including **2,007** images and **81** videos. The data was divided into training, validation, and testing sets with a split of approximately 66%, 19%, and 15%, respectively.

The dataset mainly focuses on fatigue detection, with 1,853 fatigue-related images and 76 fatigue videos, while gesture data includes 184 images and 46 activation videos. This imbalance reflects the importance of accurately detecting fatigue conditions in the system.

4. System Components:

4.1. Constants:

The constants file is used to store key parameters such as thresholds for fatigue detection and gesture recognition. These include values related to the **Eye Aspect Ratio (EAR)**, **Mouth Aspect Ratio (MAR)**, and frame-based conditions. This makes it easy to adjust the values without changing the main code.

4.2: Fatigue Detection Modules:

The fatigue detection system uses visual features from the driver's face, such as the eyes, mouth, and head position, to detect signs of fatigue.

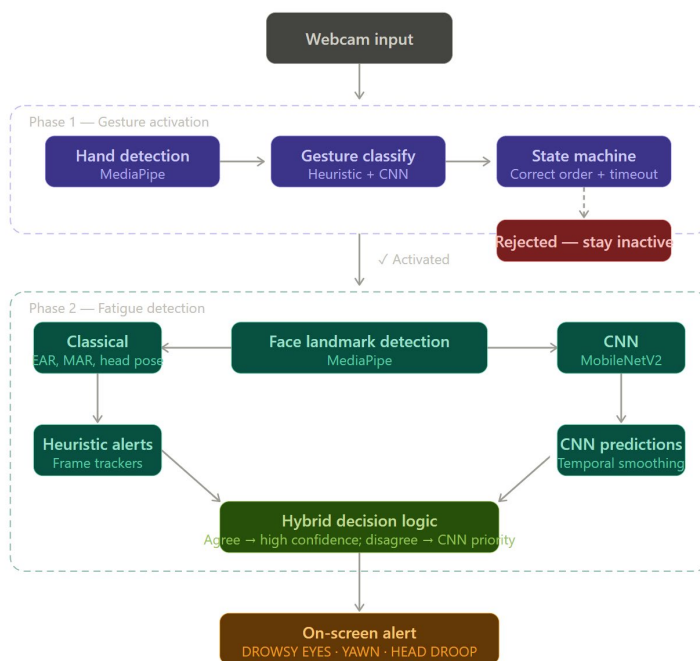
Eye closure is detected using the Eye Aspect Ratio (EAR), while yawning is detected using the Mouth Aspect Ratio (MAR). A calibration step is applied for MAR to adapt the threshold based on each user's natural mouth position.

Head movement is also analysed to detect fatigue-related behaviours such as head drooping and head turning. These signals are tracked across multiple frames using dedicated tracker classes to improve stability and reduce noise.

4.3: Gesture Detection:

The system displays these features in real time and triggers alerts such as “DROWSY EYES”, “YAWN”, and “HEAD DROOP” when abnormal patterns are detected.

Gesture detection is done using hand features extracted from MediaPipe. The system recognizes gestures such as thumbs up and open palm based on the relative positions of finger joints. A gesture sequence (thumbs up followed by open palm) is required to activate the system. This ensures that fatigue detection does not run continuously and prevents accidental activation. During development, we adjusted the gesture logic to make it more reliable. At first, it was either too strict or too easy to trigger, so we improved the rules to make it more balanced.



4.4: Machine Learning Models:

In addition to the classical approach, deep learning models were trained to improve detection accuracy. Two convolutional neural network (CNN) models based on MobileNetV2 were used: one for gesture classification and one for fatigue detection.

The gesture model classifies three classes: open palm, thumbs up, and wrong gesture. The fatigue model classifies five states: alert, eyes closed, head pose, tired expression, and yawn. The models were trained using transfer learning with pretrained ImageNet weights. Data augmentation techniques such as flipping, brightness adjustment, and rotation were applied to improve generalization. Training was performed using the Adam optimizer with focal loss to handle class imbalance.

4.5: Hybrid System:

The system uses a hybrid approach that combines classical methods and deep learning. The classical method relies on geometric features such as EAR, MAR, and head movement, while the CNN analyzes visual patterns from the driver's face. The final decision is made by combining both approaches. When both methods agree, the system produces a high-confidence alert. In cases of disagreement, the system prioritizes CNN predictions for appearance-based features such as yawning and eye closure and relies on heuristic methods for geometric features such as head movement. A temporal smoothing mechanism is also applied to CNN predictions by averaging outputs over multiple frames. This reduces noise and prevents false alerts caused by sudden changes in a single frame.

5. Implementation:

The system is implemented in Python using OpenCV and MediaPipe for real-time video processing. The main program captures video frames from the webcam and processes them continuously in a loop. For example, the system reads frames using:

```
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
```

The system operates in two main phases: gesture-based activation and fatigue detection. In the first phase, the system remains inactive and waits for a gesture sequence. Hand detection is performed using MediaPipe, and gestures are classified based on finger positions. The detected gesture is passed to a state machine that checks whether the correct sequence is completed:

```
gesture = heuristic_clf.classify(landmarks)
state_machine.update(gesture)
```

This ensures that the system only activates after detecting the required gestures in the correct order and within a limited time.

Once activated, the system enters the fatigue detection phase. Facial features are detected using MediaPipe, and different indicators are analyzed. Eye closure is detected using the Eye Aspect Ratio (EAR), which is calculated and updated as follows:

```
avg_ear = (compute_ear(left_eye) + compute_ear(right_eye)) / 2.0

eye_tracker.update(avg_ear)
```

Similarly, yawning is detected using the Mouth Aspect Ratio (MAR), where the system checks if the mouth remains open for multiple frames:

```
mar_value = compute_mar(mouth_pts)

yawn_tracker.update(mar_value)
```

Head movement is also tracked to detect fatigue-related behaviors such as head drooping and turning:

```
head_tracker.update(nose_y, face_height)
head_tracker.update_yaw(*yaw_pts)
```

These features are monitored across multiple frames to improve stability and avoid false detections.

The system also supports a hybrid mode, where deep learning models are used alongside the classical approach. A convolutional neural network (CNN) analyzes the driver's face and provides additional predictions. These predictions are combined with the heuristic results to generate the final output:

```
final_alerts, hybrid_source = hybrid_fatigue_decision(
    heuristic_alerts, cnn_class, cnn_conf
)
```

To further improve performance, smoothing techniques are applied to reduce sudden fluctuations. For example, CNN predictions are averaged over multiple frames to prevent noise from triggering false alerts:

```
cnn_class, cnn_conf = fatigue_smoother.update(raw_proba)
```

In addition, threshold values and frame-based conditions are used to ensure that fatigue is only detected when a behavior persists for a short period of time, making the system more reliable in real-time scenarios.

6. Experimental Setup:

The system was tested in a realistic environment using a webcam positioned in front of the user to simulate a dashboard camera. The testing was conducted inside a parked car under daylight conditions to reflect a real driving scenario. The system processes video frames in real time and displays the results directly on the screen, including detected gestures and fatigue alerts.

Different scenarios were tested to evaluate the system's performance, including normal driving behavior, slight movements, and clear fatigue actions such as yawning, head tilting, and prolonged eye closure. These tests allowed us to observe how the system behaves under different conditions and to evaluate its stability and responsiveness in real-time operation.

7. Results:

The system was evaluated across a range of test scenarios inside a parked vehicle under daylight conditions. Results are reported for both the gesture-based activation and the fatigue detection modules.

Gesture-Based Activation: The gesture-based activation worked correctly using the two-gesture sequence (thumbs up followed by open palm). The sequence was successfully activated within the allowed time window across repeated trials, with no accidental activations observed during normal hand movement. The state machine correctly rejected out-of-order and incomplete gesture attempts in all cases tested.

Fatigue Detection (Classical Method): Eye closure was detected using an EAR threshold of 0.25, with alerts triggered after the condition persisted for 20 consecutive frames. Yawning was detected using a calibrated MAR threshold set relative to each user's baseline, with alerts triggered after 15 frames. Head drooping was flagged when the nose position dropped more than 15% of face height from the baseline, and head turning was flagged beyond a yaw deviation threshold. These thresholds were tuned iteratively during testing to balance sensitivity and false positive rate.

Fatigue Detection(CNN and Hybrid): The MobileNetV2-based fatigue model, trained on our custom dataset with a 66/19/15% train/validation/test split, achieved reliable classification across the five target states (alert, eyes closed, head pose, tired expression, and yawn) in controlled lighting conditions. Temporal smoothing over a sliding window of frames further stabilised CNN outputs and reduced single-frame false alerts. The hybrid decision logic, which combines classical geometric features with CNN predictions, improved overall robustness compared to either method alone. In particular, the CNN helped in cases where geometric thresholds alone were insufficient, such as partial yawns and subtle tired expressions.

Discussion: Initially, the system was too sensitive to yawning and head movement, and gesture detection was not always consistent. After iterative threshold tuning and gesture logic refinement, the system became more stable. Frame-based persistence conditions were key to eliminating transient false positives. The results observed during testing were consistent with what is shown in the demonstration video. Overall, the final system reliably detected fatigue-related behaviours in real-time during all test scenarios conducted.

8. Limitations:

Although the system performs well in most scenarios, there are still some limitations. The performance can be affected by lighting conditions, as poor lighting may reduce the accuracy of face and hand detection. In addition, if the head is turned too much or part of the face is covered, the system may have trouble detecting fatigue correctly.

Gesture detection may also be influenced by hand position and orientation, especially if the hand is not clearly visible to the camera. Finally, the CNN model may sometimes make mistakes, especially when different fatigue behaviors look similar.

9. Conclusion:

In this project, we developed a driver fatigue detection system using computer vision techniques combined with a gesture-based activation mechanism. The system is designed to remain inactive until the correct gesture sequence is performed, which helps avoid unnecessary processing and makes it more practical to use. The system uses both classical methods and deep learning models to improve accuracy.

Overall, the combination of gesture activation and hybrid fatigue detection resulted in a system that is both responsive and stable. This project demonstrates how computer vision can be used to improve driver safety and reduce the risk of accidents caused by drowsy driving.

10. Group Contributions:

Iterative Code Development: The codebase was developed iteratively, with each member building upon the previous version and sharing updates with the group. Sami took an initial pass at implementing core detection logic and shared a working version with the team. Fairouz then reviewed and extended the code, refining detection thresholds and improving the gesture recognition pipeline. Daniel worked on further improvements, addressing accuracy issues and restructuring parts of the codebase for clarity and performance. Mona reviewed the latest version, tested it under different conditions, and made additional fixes to reduce false detections. Qamar then built on top of the existing structure, adding further refinements and integrating components more cohesively. This cycle of sharing, reviewing, and improving continued throughout the project, ensuring that all members had an understanding of the full system.